

M02 / PROTECTED ANSWERS

M02 M03 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first

7

YOUR STANDALONE STUDY BOOK

7

Open the matching review after saving your attempt.

7

Contents

7

DE02-01 | Database, table, row, column and query mental model

8

First practice: explained review

8

Changed retry: explained review

8

DE02-02 | SELECT, aliases and LIMIT

8

First practice: explained review

8

Changed retry: explained review

9

DE02-03 | WHERE and comparison operators

9

First practice: explained review

9

Changed retry: explained review

9

DE02-04 | AND, OR, NOT and parentheses

9

First practice: explained review

9

Changed retry: explained review

10

DE02-05 | ORDER BY, DISTINCT and result inspection

10

First practice: explained review

10

Changed retry: explained review

10

DE02-06 | NULL, IS NULL and COALESCE

10

First practice: explained review

10

Changed retry: explained review	11
DE02-07 Arithmetic expressions and safe calculations	11
First practice: explained review	11
Changed retry: explained review	11
DE02-08 Aggregate functions	11
First practice: explained review	11
Changed retry: explained review	12
DE02-09 GROUP BY and grouped grain	12
First practice: explained review	12
Changed retry: explained review	12
DE02-10 HAVING after aggregation	12
First practice: explained review	12
Changed retry: explained review	13
DE02-11 CASE for business categories	13
First practice: explained review	13
Changed retry: explained review	13
DE02-12 String and date basics	13
First practice: explained review	13
Changed retry: explained review	14
DE02-13 Simple subqueries and IN/EXISTS awareness	14
First practice: explained review	14
Changed retry: explained review	14
DE02-14 Debugging and validation workflow	15

First practice: explained review	15
Changed retry: explained review	15
DE03-01 INNER JOIN and LEFT JOIN	15
First practice: explained review	15
Changed retry: explained review	16
DE03-02 Cardinality, row multiplication and join validation	16
First practice: explained review	16
Changed retry: explained review	16
DE03-03 UNION and UNION ALL	16
First practice: explained review	16
Changed retry: explained review	17
DE03-04 String functions for cleaning and standardization	17
First practice: explained review	17
Changed retry: explained review	17
DE03-05 Dates, timestamps and time analysis	18
First practice: explained review	18
Changed retry: explained review	18
DE03-06 Subqueries and nested logic	18
First practice: explained review	18
Changed retry: explained review	18
DE03-07 CTEs and readable multi-step SQL	19
First practice: explained review	19
Changed retry: explained review	19

DE03-08 Window functions and PARTITION BY	19
First practice: explained review	19
Changed retry: explained review	19
DE03-09 ROW_NUMBER, ranking and LAG/LEAD	20
First practice: explained review	20
Changed retry: explained review	20
DE03-10 Advanced aggregation and multi-grain reasoning	21
First practice: explained review	21
Changed retry: explained review	21
DE03-11 PostgreSQL setup, schemas and dialect awareness	21
First practice: explained review	21
Changed retry: explained review	22
DE03-12 Constraints and transactions	22
First practice: explained review	22
Changed retry: explained review	22
DE03-13 Indexes and EXPLAIN basics	23
First practice: explained review	23
Changed retry: explained review	23
DE03-14 Views and reusable database logic	23
First practice: explained review	23
Changed retry: explained review	23
DE03-15 Integrated unseen SQL challenge	24
First practice: explained review	24

Changed retry: explained review	25
DE03-16 Timed SQL interview set and explanation	25
First practice: explained review	25
Changed retry: explained review	26

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

YOUR STANDALONE STUDY BOOK

Open the matching review after saving your attempt.

These explanations help you identify and repair mistakes. Keep the first attempt, compare its reasoning and actual output, then solve the changed retry independently. SQL answer files are divided into blocks so you can execute the matching example deliberately. Some temporary-object exercises depend on earlier blocks within that lesson. The learner book supplies that order.

Contents

DE02-01 Database, table, row, column and query mental model	4
DE02-02 SELECT, aliases and LIMIT	5
DE02-03 WHERE and comparison operators	6
DE02-04 AND, OR, NOT and parentheses	7
DE02-05 ORDER BY, DISTINCT and result inspection	8
DE02-06 NULL, IS NULL and COALESCE	9
DE02-07 Arithmetic expressions and safe calculations	10
DE02-08 Aggregate functions	11
DE02-09 GROUP BY and grouped grain	12
DE02-10 HAVING after aggregation	13
DE02-11 CASE for business categories	14
DE02-12 String and date basics	15
DE02-13 Simple subqueries and IN/EXISTS awareness	16
DE02-14 Debugging and validation workflow	17
DE03-01 INNER JOIN and LEFT JOIN	18
DE03-02 Cardinality, row multiplication and join validation	19
DE03-03 UNION and UNION ALL	20
DE03-04 String functions for cleaning and standardization	21
DE03-05 Dates, timestamps and time analysis	22

DE03-06 Subqueries and nested logic	23
DE03-07 CTEs and readable multi-step SQL	24
DE03-08 Window functions and PARTITION BY	25
DE03-09 ROW_NUMBER, ranking and LAG/LEAD	26
DE03-10 Advanced aggregation and multi-grain reasoning	27
DE03-11 PostgreSQL setup, schemas and dialect awareness	28
DE03-12 Constraints and transactions	29
DE03-13 Indexes and EXPLAIN basics	30
DE03-14 Views and reusable database logic	31
DE03-15 Integrated unseen SQL challenge	32
DE03-16 Timed SQL interview set and explanation	34

DE02-01 | Database, table, row, column and query mental model

First practice: explained review

A correct request is:

```
SELECT order_id, item
FROM course.practice_sales
ORDER BY order_id;
```

The four results are P101 Pen, P102 Notebook, P103 Folder and P104 Bag. This result has four rows and two columns. The underlying table retains its other columns. FROM chooses the practice table; using course.sales would execute successfully but answer the wrong question. That is a meaning error, not a SQL syntax error.

Changed retry: explained review

```
SELECT order_id, quantity, unit_price
FROM course.retry_sales
ORDER BY order_id;
```

Rows are R201/3/25.00, R202/2/10.00, R203/3/25.00 and R204/2/15.00. IDs distinguish records; quantities measure units and support arithmetic. Four rows and three selected columns should appear. The correct input table matters as much as the selected columns.

DE02-02 | SELECT, aliases and LIMIT

First practice: explained review

```
SELECT order_id AS reference, item AS product_name
FROM course.practice_sales
ORDER BY order_id
LIMIT 2;
```

P101 Pen and P102 Notebook appear. The result proves that this query returned two rows. It does not prove there are only two source records; the supplied practice table has four. A LIMIT belongs to a request, not to a source deletion operation.

Changed retry: explained review

```
SELECT order_id AS reference, item
FROM course.retry_sales
ORDER BY order_id DESC
LIMIT 2;
```

R204 Ruler then R203 Folder should appear. A new query against course.retry_sales still names order_id. The alias did not rename the stored field. Descending sort changes what “first two” means for the limited output.

DE02-03 | WHERE and comparison operators

First practice: explained review

```
SELECT order_id, item, unit_price
FROM course.practice_sales
WHERE unit_price > 30
ORDER BY order_id;
```

P102 Notebook 40.00 and P104 Bag 120.00 remain. P103 Folder at 30.00 is excluded. P101 Pen at 9.00 is also excluded. Returning three records usually means you used >= rather than >. Inspect the actual rule instead of merely changing an expected row count.

Changed retry: explained review

```
SELECT order_id, ordered_on
FROM course.retry_sales
WHERE ordered_on < DATE '2026-11-02'
ORDER BY order_id;
```

R201 and R202, both November 1, remain. “Before” excludes the boundary date. R203 on November 2 would appear with <=, which would answer a different question.

DE02-04 | AND, OR, NOT and parentheses

First practice: explained review

```
SELECT order_id, category, quantity
FROM course.practice_sales
WHERE (category = 'Paper'
      OR category = 'Bags')
      AND quantity = 1
ORDER BY order_id;
```

P103 Paper/1 and P104 Bags/1 remain. P102 Paper/2 fails the quantity rule. Without the intended grouping, the Paper branch can admit P102 and make the result wrong even though the query runs.

Changed retry: explained review

```
SELECT order_id, category, quantity
FROM course.retry_sales
WHERE NOT (category = 'Paper')
      AND quantity >= 2
ORDER BY order_id;
```

R202 Writing/2 and R204 Tools/2 qualify. Both R201 and R203 are Paper and fail the first test. Parentheses around the NOT operand make the intended scope visible.

DE02-05 | ORDER BY, DISTINCT and result inspection

First practice: explained review

```
SELECT DISTINCT category
FROM course.practice_sales
ORDER BY category;
```

```
SELECT order_id, unit_price
FROM course.practice_sales
ORDER BY unit_price DESC, order_id
LIMIT 2;
```

The category list is Bags, Paper, Writing: three names for four records because two records are Paper. The price preview is P104/120.00 then P102/40.00. Collapsing repeated categories is exactly the first question's intended operation; it does not establish that any sale was duplicated.

Changed retry: explained review

```
SELECT DISTINCT item, unit_price
FROM course.retry_sales
ORDER BY item, unit_price;
```

```
SELECT order_id, quantity
FROM course.retry_sales
ORDER BY quantity DESC, order_id
LIMIT 2;
```

The four item/price combinations are Folder/25, Notebook/25, Pen/10 and Ruler/15. Folder and Notebook remain separate because item differs even though price matches. The preview is R201/3 then R203/3, ordered by their IDs within the quantity tie.

DE02-06 | NULL, IS NULL and COALESCE

First practice: explained review

```
SELECT order_id
FROM course.practice_sales
WHERE is_paid IS NULL
ORDER BY order_id;
```

```
SELECT order_id
FROM course.practice_sales
WHERE is_paid = FALSE
ORDER BY order_id;
```

```
SELECT order_id, discount_pct,
      COALESCE(discount_pct, 0) AS estimated_discount_pct
FROM course.practice_sales
```

```
WHERE discount_pct IS NULL;
```

Unknown payment is P104; known unpaid is P103. P103 has the missing discount and a provisional zero in the second displayed discount field. The source remains unknown. If you put P104 in the known-unpaid group, repair the meaning before accepting a matching total elsewhere.

Changed retry: explained review

```
SELECT order_id
FROM course.retry_sales
WHERE customer_id IS NULL
ORDER BY order_id;
```

```
SELECT order_id
FROM course.retry_sales
WHERE discount_pct IS NULL
ORDER BY order_id;
```

R204 lacks a customer ID; R203 lacks a discount. Missingness belongs to individual fields. A row may contain known quantities, unknown customer identity and a known zero discount at the same time. Missing one field does not make every other value unusable.

DE02-07 | Arithmetic expressions and safe calculations

First practice: explained review

```
SELECT order_id, quantity * unit_price AS gross_amount
FROM course.practice_sales
ORDER BY order_id;
```

```
SELECT order_id, quantity * unit_price * (1 - discount_pct) AS net
FROM course.practice_sales
WHERE order_id = 'P102';
```

Gross amounts are P101 18, P102 80, P103 30 and P104 120. P102 net is 72: two notebooks at 40 make 80, and a 10% discount leaves 90% of that amount. Multiplying by discount_pct alone would return the discount amount, 8, not the remaining payable amount.

Changed retry: explained review

```
SELECT order_id, quantity * unit_price AS gross_amount
FROM course.retry_sales
ORDER BY order_id;
```

```
SELECT 3.0 / NULLIF(0, 0) AS return_rate;
```

Gross amounts are 75, 20, 75 and 30. The return_rate is NULL because the denominator is zero. That is an undefined ratio, not evidence that no returns occurred. The numerator explicitly says three units were returned.

DE02-08 | Aggregate functions

First practice: explained review

```
SELECT COUNT(*) AS sale_rows, COUNT(is_paid) AS known_payment_rows,
       SUM(quantity * unit_price) AS gross_total, AVG(quantity * unit_price) AS mean_gross
FROM course.practice_sales;
```

```
SELECT COUNT(*) AS unpaid_rows, SUM(quantity * unit_price) AS unpaid_gross
FROM course.practice_sales
WHERE is_paid = FALSE;
```

The first row is 4, 3, 248 and 62. The second is 1 and 30, from P103. Each query returns one summary for its qualifying set, not one row per sale. P104's unknown Boolean is not counted among known payments or unpaid payments.

Changed retry: explained review

```
SELECT COUNT(*) AS sale_rows, COUNT(is_paid) AS known_payment_rows,
      SUM(quantity) AS units, SUM(quantity * unit_price) AS gross_total,
      AVG(quantity * unit_price) AS mean_sale
FROM course.retry_sales;
```

The values are 4, 3, 10, 200 and 50. Mean sale amount divides 200 by four sales. Dividing by ten units would answer the different question of revenue per unit: 20. Names and units prevent these measures being mistaken for each other.

DE02-09 | GROUP BY and grouped grain

First practice: explained review

```
SELECT category, COUNT(*) AS sale_rows,
      SUM(quantity * unit_price) AS gross_total
FROM course.practice_sales
GROUP BY category
ORDER BY category;
```

Bags is 1/120, Paper 2/110 and Writing 1/18. P102 contributes 80 and P103 contributes 30 to Paper. Counts total four and amounts total 248. Three group rows are correct even though four sales exist. Each row now represents a category, not a sales record.

Changed retry: explained review

```
SELECT ordered_on, COUNT(*) AS sale_rows, SUM(quantity) AS units,
      SUM(quantity * unit_price) AS gross_total
FROM course.retry_sales
GROUP BY ordered_on
ORDER BY ordered_on;
```

November 1 is 2 rows/5 units/95; November 2 is 1/3/75; November 3 is 1/2/30. Across the three day summaries there are four source rows, ten units and 200 gross amount. Source record count, quantity and currency have different units and should remain separately named.

DE02-10 | HAVING after aggregation

First practice: explained review

```
SELECT category, SUM(quantity * unit_price) AS gross_total
FROM course.practice_sales
GROUP BY category
HAVING SUM(quantity * unit_price) > 100
ORDER BY category;
```

Bags 120 and Paper 110 qualify. Paper contains 80 and 30: neither sale alone exceeds 100. A WHERE line_amount > 100 would retain only the bag sale and lose the intended Paper group.

Writing 18 correctly fails the group threshold.

Changed retry: explained review

```
SELECT category, COUNT(*) AS sale_rows,
       SUM(quantity * unit_price) AS gross_total
FROM course.retry_sales
GROUP BY category
HAVING COUNT(*) >= 2
ORDER BY category;
```

Only Paper remains, with two rows and gross 150. Writing and Tools each have one row. The number of records in a category is a property of the group; it is not an original field on each sale.

DE02-11 | CASE for business categories

First practice: explained review

```
SELECT order_id,
       CASE WHEN is_paid = TRUE THEN 'Paid' WHEN is_paid = FALSE THEN 'Unpaid' ELSE 'Unknown'
       END AS payment_group
FROM course.practice_sales
ORDER BY order_id;
```

```
SELECT
  SUM(CASE WHEN is_paid = TRUE
           THEN quantity * unit_price ELSE 0 END) AS paid,
  SUM(CASE WHEN is_paid = FALSE
           THEN quantity * unit_price ELSE 0 END) AS unpaid,
  SUM(CASE WHEN is_paid IS NULL
           THEN quantity * unit_price ELSE 0 END) AS unknown
FROM course.practice_sales;
```

P101 and P102 are Paid, P103 Unpaid, P104 Unknown. The amount summary is 98, 30 and 120; their total is 248. No missing payment was labelled unpaid, and each sale contributed to one bucket only.

Changed retry: explained review

```
SELECT order_id,
       CASE WHEN quantity * unit_price >= 75 THEN 'High' WHEN quantity * unit_price >= 30 THEN 'Mid' ELSE 'Low'
       END AS amount_group
FROM course.retry_sales
ORDER BY order_id;
```

R201 and R203 are High at exactly 75, R204 Mid at exactly 30, and R202 Low at 20. These boundary cases are more useful than checking only a value far from every threshold. First matching branch wins.

DE02-12 | String and date basics

First practice: explained review

```
SELECT order_id, LOWER(item) AS item_lower,
       ordered_on + 3 AS follow_up_date
FROM course.practice_sales
ORDER BY order_id;
```

```
SELECT order_id
FROM course.practice_sales
```

```
WHERE ordered_on >= DATE '2026-10-01'  
      AND ordered_on < DATE '2026-10-02'  
ORDER BY order_id;
```

The lowercase items are pen, notebook, folder and bag. Follow-up dates are October 4, 4, 5 and 6. The one-day selection returns P101 and P102. The October 2 boundary belongs to the following day and is excluded by <.

Changed retry: explained review

```
SELECT item, UPPER(item) AS item_upper,  
       EXTRACT(MONTH  
             FROM ordered_on) AS month_number, ordered_on + 7 AS follow_up_date  
FROM course.retry_sales  
WHERE order_id = 'R203';
```

Expected Folder, FOLDER, 11 and 2026-11-09. UPPER transforms the result expression only; it does not rewrite the source item. The stored date is November 2 and adding seven days produces November 9.

DE02-13 | Simple subqueries and IN/EXISTS awareness

First practice: explained review

```
SELECT order_id, quantity * unit_price AS gross_amount  
FROM course.practice_sales  
WHERE quantity * unit_price >  
      (SELECT AVG(quantity * unit_price)  
       FROM course.practice_sales)  
ORDER BY order_id;  
  
SELECT EXISTS  
  (SELECT 1  
   FROM course.practice_sales  
   WHERE customer_id IS NULL) AS has_missing_customer;
```

The mean is 62, so P102/80 and P104/120 qualify. Existence is true because P104 lacks a customer ID. Comparing with the guided table's 76.75 average happens to produce the same two IDs here, so a matching output alone would not prove that you used the right population. Inspect the inner query and its 62 threshold as well.

Changed retry: explained review

```
SELECT order_id, quantity * unit_price AS gross_amount  
FROM course.retry_sales  
WHERE quantity * unit_price >=  
      (SELECT AVG(quantity * unit_price)  
       FROM course.retry_sales)  
ORDER BY order_id;
```

The threshold is 50, so R201/75 and R203/75 qualify. EXISTS answers true/false about the presence of rows; it does not supply the numeric mean. The subquery's output shape must fit the outer question.

DE02-14 | Debugging and validation workflow

First practice: explained review

```
SELECT category, SUM(quantity * unit_price) AS gross_total
FROM course.practice_sales
WHERE ordered_on >= DATE '2026-10-01'
      AND ordered_on < DATE '2026-10-03'
GROUP BY category
HAVING SUM(quantity * unit_price) >= 30
ORDER BY category;
```

```
SELECT COUNT(*) AS selected_rows, SUM(quantity * unit_price) AS selected_gross
FROM course.practice_sales
WHERE ordered_on >= DATE '2026-10-01'
      AND ordered_on < DATE '2026-10-03';
```

Before HAVING, the grouped result is Paper 110 and Writing 18. After it, only Paper 110 remains. The selected source contains three records totalling 128. The 18 difference is the intentionally excluded Writing group, not missing data. The half-open end of October 3 includes both requested dates and excludes P104 on October 3.

Changed retry: explained review

```
SELECT
  CASE WHEN is_paid = TRUE THEN 'Paid'
        WHEN is_paid = FALSE THEN 'Unpaid'
        ELSE 'Unknown' END AS payment_group,
  COUNT(*) AS sale_rows,
  SUM(quantity * unit_price) AS gross_total
FROM course.retry_sales
GROUP BY
  CASE WHEN is_paid = TRUE THEN 'Paid'
        WHEN is_paid = FALSE THEN 'Unpaid'
        ELSE 'Unknown' END
ORDER BY payment_group;
```

Paid is 2/150, Unknown 1/30 and Unpaid 1/20. Counts total four; amounts total 200. Labelling the unknown row as unpaid would show 50 unpaid, mixing one known unpaid sale with one whose status has not been recorded. A reconciled grand total would still be 200, so total agreement alone would miss that meaning error.

DE03-01 | INNER JOIN and LEFT JOIN

First practice: explained review

```
SELECT c.customer_id, c.customer_name, o.order_id
FROM course.customers AS c
LEFT JOIN course.orders AS o ON c.customer_id = o.customer_id
ORDER BY c.customer_id, o.order_id;
```

There are six output rows: Asha/O101, Asha/O103, Ravi/O102, Mina/O104, Iqbal/O105 and Tara/NULL. The anonymous order O106 does not become a new customer record; the left side consists of existing customers. Asha's two output rows are legitimate one-to-many matches.

Changed retry: explained review

```
SELECT o.order_id, c.customer_name
FROM course.orders AS o
LEFT JOIN course.customers AS c ON o.customer_id = c.customer_id
AND c.region = 'North'
ORDER BY o.order_id;
```

Six rows remain; only O105 has Iqbal's name. A WHERE restriction would keep just O105 because it removes null-extended and non-North matches after joining. The placement of a right-side rule can change which left records survive.

DE03-02 | Cardinality, row multiplication and join validation

First practice: explained review

```
SELECT SUM(i.quantity * i.unit_price) AS item_total,
       SUM(i.quantity * i.unit_price + o.shipping_fee) AS wrong_order_total
FROM course.orders AS o
JOIN course.order_items AS i ON o.order_id = i.order_id
WHERE o.order_id = 'O105';
```

Items are Notebook 60 and Pen 50, total 110. The correct order amount is $110 + 5 = 115$. The deliberately wrong joined sum is 120 because two line rows each add the same fee 5. The intermediate grain is one item line, so an order-level charge repeats there.

Changed retry: explained review

```
SELECT COUNT(*) AS result_rows, COUNT(i.line_no) AS matched_item_lines,
       COUNT(DISTINCT o.order_id) AS represented_orders
FROM course.orders AS o
LEFT JOIN course.order_items AS i ON o.order_id = i.order_id;
```

Results are 9, 8 and 6. O106 contributes a preserved result row with a null line_no, so COUNT(*) includes it while COUNT(i.line_no) does not. Three orders have multiple line rows, so there are more joined rows than distinct orders. Explain all three counts before accepting the join.

DE03-03 | UNION and UNION ALL

First practice: explained review

```
SELECT category
FROM course.practice_sales
WHERE ordered_on = DATE '2026-10-01'
UNION ALL
SELECT category
FROM course.practice_sales
WHERE ordered_on = DATE '2026-10-02'
ORDER BY category;

SELECT category
FROM course.practice_sales
WHERE ordered_on = DATE '2026-10-01'
UNION
SELECT category
FROM course.practice_sales
WHERE ordered_on = DATE '2026-10-02'
ORDER BY category;
```

UNION ALL returns Paper, Paper, Writing: three occurrences from three sales. UNION returns Paper and Writing: two unique selected values. P102 and P103 are separate Paper sales; collapsing their category labels is appropriate only for a question about categories.

Changed retry: explained review

```
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-01' INTERSECT
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-02';
```

```
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-01'
EXCEPT
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-02';
```

The intersection is Paper. The first-minus-second category is Writing. Reversing the EXCEPT branches changes the question and, here, produces no rows because November 2 contains only Paper, which also appears on November 1.

DE03-04 | String functions for cleaning and standardization

First practice: explained review

```
SELECT source_id, item_raw
FROM course.raw_labels
WHERE NULLIF(LOWER(TRIM(item_raw)), '') IS NULL
ORDER BY source_id;

SELECT COUNT(DISTINCT NULLIF(LOWER(TRIM(item_raw)),
                             '')) AS distinct_clean_labels
FROM course.raw_labels;
```

C and D are the missing cleaned labels; C began as empty text and D as NULL. There are three distinct non-null clean values: pen, note book and pen blue. Four usable rows produce only three unique labels because A and B both become pen. That does not justify deleting one source row without understanding what a row represents.

Changed retry: explained review

```
SELECT ' Bag ' AS raw_value, NULLIF(LOWER(TRIM(' Bag ')),
                                     '') AS clean_value,
       LENGTH(NULLIF(LOWER(TRIM(' Bag ')), '')) AS clean_length
UNION ALL
SELECT 'BAG', NULLIF(LOWER(TRIM('BAG')), ''),
       LENGTH(NULLIF(LOWER(TRIM('BAG')), ''))
UNION ALL
SELECT ' ', NULLIF(LOWER(TRIM(' ')), ''),
       LENGTH(NULLIF(LOWER(TRIM(' ')), ''));
```

The first two cleaned values are bag with length 3. The third clean value and clean length are both NULL because the rule converts empty cleaned text to missing. If you wanted the length of the intermediate trimmed string, it would be zero; that is a different expression and a useful

debugging distinction.

DE03-05 | Dates, timestamps and time analysis

First practice: explained review

```
SELECT event_id
FROM course.events
WHERE happened_at >= TIMESTAMPTZ '2026-09-02 00:00:00+05:30'
      AND happened_at < TIMESTAMPTZ '2026-09-03 00:00:00+05:30'
ORDER BY happened_at;
```

Only T3 qualifies. It equals the new day's inclusive beginning and the previous day's exclusive end. Adjacent half-open day windows therefore include it once, rather than twice or not at all.

Changed retry: explained review

```
SELECT event_id
FROM course.events
WHERE happened_at >= TIMESTAMPTZ '2026-09-01 00:00:00+00'
      AND happened_at < TIMESTAMPTZ '2026-09-02 00:00:00+00'
ORDER BY happened_at;
```

T2 and T3 qualify. T1 is August 31 in UTC, while T3 is still September 1 in UTC. The source instants did not change; the definition of the reporting day changed. A date total without its timezone convention can be ambiguous.

DE03-06 | Subqueries and nested logic

First practice: explained review

```
SELECT c.customer_id, c.customer_name
FROM course.customers AS c
WHERE NOT EXISTS
  (SELECT 1
   FROM course.orders AS o
   WHERE o.customer_id = c.customer_id
        AND o.is_paid = TRUE)
ORDER BY c.customer_id;
```

C02 Ravi, C03 Mina and C05 Tara remain. Mina's recorded order has unknown payment. The result says only that no matched record establishes a TRUE payment in these inputs. Labelling all three “unpaid” would overstate what the data proves.

Changed retry: explained review

```
SELECT c.customer_id, c.customer_name
FROM course.customers AS c
WHERE
  (SELECT COUNT(*)
   FROM course.orders AS o
   WHERE o.customer_id = c.customer_id) >= 2
ORDER BY c.customer_id;
```

Only C01 Asha qualifies. EXISTS distinguishes zero from at least one; the condition here distinguishes fewer than two from at least two. The subquery must supply the kind of evidence the outer test needs.

DE03-07 | CTEs and readable multi-step SQL

First practice: explained review

```
WITH item_totals AS
  (SELECT order_id, SUM(quantity * unit_price) AS item_gross
   FROM course.order_items
   GROUP BY order_id)
SELECT o.order_id,
       COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross
FROM course.orders AS o
LEFT JOIN item_totals AS i ON o.order_id = i.order_id
WHERE o.is_paid = TRUE
ORDER BY o.order_id;
```

O101 149, O103 100 and O105 115 appear, total 364. Their item gross is $129 + 100 + 110 = 339$, and their fees are $20 + 0 + 5 = 25$. Both descriptions reconcile to 364. A source-wide 614 item total would be the wrong comparison scope for this paid-only result.

Changed retry: explained review

```
WITH item_totals AS
  (SELECT order_id, SUM(quantity * unit_price) AS item_gross
   FROM course.order_items
   GROUP BY order_id)
SELECT o.order_id
FROM course.orders AS o
LEFT JOIN item_totals AS i ON o.order_id = i.order_id
WHERE i.order_id IS NULL
ORDER BY o.order_id;
```

Only O106 appears. A missing right-side key proves no matching summary row exists here. An amount of zero could also come from actual zero-price lines, so the numeric amount alone is not a reliable existence test. Preserve the distinction between “no rows” and “rows summing to zero.”

DE03-08 | Window functions and PARTITION BY

First practice: explained review

```
SELECT order_id, category, quantity * unit_price AS gross_amount,
       SUM(quantity * unit_price) OVER (PARTITION BY category) AS category_gross
FROM course.practice_sales
ORDER BY order_id;
```

P101 shows 18/18, P102 80/110, P103 30/110 and P104 120/120 for gross/category gross. Paper's summary repeats because two Paper detail rows remain. Adding category_gross would yield 358, not the source total 248. The repeated aggregate is a context column, not an additive detail measure.

Changed retry: explained review

```
SELECT order_id,
       SUM(quantity * unit_price) OVER (
                                     ORDER BY ordered_on,
                                     order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_gross
FROM course.retry_sales
ORDER BY ordered_on, order_id;
```

R201 75, R202 95, R203 170 and R204 200. The first day's two records accumulate one at a time in ID order. The final running total reconciles to the ungrouped source total, while each intermediate number follows the specified ordering and frame.

DE03-09 | ROW_NUMBER, ranking and LAG/LEAD

First practice: explained review

```
WITH ranked AS
  (SELECT customer_id, order_id, ordered_on,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                           ORDER BY ordered_on DESC,
                           order_id DESC) AS rn
   FROM course.practice_sales)
SELECT customer_id, order_id, ordered_on
FROM ranked
WHERE rn = 1
ORDER BY customer_id NULLS LAST;
```

C10 gets P103 on October 2, C11 gets P102 on October 1, and the unknown bucket gets P104 on October 3. If several unidentified customers existed, partitioning all NULLs together would not establish which record is latest for each real person. The exercise preserves a data-quality bucket; it does not solve missing identity.

Changed retry: explained review

```
WITH previous AS
  (SELECT order_id, ordered_on, quantity * unit_price AS amount,
         LAG(quantity * unit_price) OVER (
                                   ORDER BY ordered_on,
                                   order_id) AS previous_amount
   FROM course.retry_sales)
SELECT order_id, amount, previous_amount,
       amount - previous_amount AS change_from_previous
FROM previous
ORDER BY ordered_on, order_id;
```

R201 is 75/NULL/NULL; R202 20/75/-55; R203 75/20/55; R204 30/75/-45. There is no earlier row for R201 in this source, so no measured change exists. Zero would mean “unchanged from a known prior value,” which has not been established.

DE03-10 | Advanced aggregation and multi-grain reasoning

First practice: explained review

```
WITH item_totals AS
  (SELECT order_id, SUM(quantity * unit_price) AS item_gross
   FROM course.order_items
   GROUP BY order_id),
  prepared AS
  (SELECT o.order_id,
   CASE WHEN o.is_paid = TRUE THEN 'Paid' WHEN o.is_paid = FALSE THEN 'Unpaid' ELSE 'Unknown'
   END AS payment_group,
   COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross
   FROM course.orders AS o
   LEFT JOIN item_totals AS i ON o.order_id = i.order_id)
SELECT payment_group, COUNT(*) AS order_count,
       SUM(order_gross) AS order_gross
FROM prepared
GROUP BY payment_group
ORDER BY payment_group;
```

Paid is three orders/364, Unknown one/265, and Unpaid two/35. The unpaid group includes O106's empty order as well as O102. Counts total six and amounts total 664. Counting only matched item-bearing orders would lose one unpaid order even though the unpaid amount still looked correct at 35.

Changed retry: explained review

```
WITH item_totals AS
  (SELECT order_id, SUM(quantity * unit_price) AS item_gross
   FROM course.order_items
   GROUP BY order_id)
SELECT COUNT(*) AS nonempty_orders, SUM(i.item_gross + o.shipping_fee) AS total_gross,
       ROUND(AVG(i.item_gross + o.shipping_fee), 2) AS avg_gross
FROM course.orders AS o
JOIN item_totals AS i ON o.order_id = i.order_id;
```

Five non-empty orders total 664, and $664 / 5$ is 132.80. The inner join is deliberate because the question explicitly requires a matching item summary. It is not a general replacement for the left join in an all-order report. The change in denominator is the entire reason the average differs.

DE03-11 | PostgreSQL setup, schemas and dialect awareness

First practice: explained review

```
SELECT COLUMN_NAME, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'course'
      AND TABLE_NAME = 'order_items'
ORDER BY ordinal_position;
```

Columns are order_id text, line_no integer, item text, quantity integer and unit_price numeric; all are non-null. The composite primary key (order_id, line_no) identifies one line. O101 legitimately occurs with line_no 1 and 2. A primary key on order_id alone would prohibit a second item line for the same order.

Changed retry: explained review

```
CREATE TEMP TABLE desk_retry (  
  reference text PRIMARY KEY,  
  units integer NOT NULL  
);  
INSERT INTO desk_retry (reference, units) VALUES ('X01', 3);  
SELECT reference, units FROM desk_retry;
```

Expect X01/3. Saving text alone does not create the object; execution does. The temporary table lasts for this session, whereas the saved .sql file remains wherever you saved it. If you need to repeat the same creation exercise, use a new session or a deliberate new temporary name.

DE03-12 | Constraints and transactions

First practice: explained review

```
BEGIN;  
CREATE TEMP TABLE payment_contract (  
  payment_id text PRIMARY KEY,  
  amount numeric(10,2) NOT NULL CHECK (amount >= 0)  
);  
INSERT INTO payment_contract VALUES ('PAY1', 50);  
COMMIT;  
BEGIN;  
UPDATE payment_contract SET amount = 60  
WHERE payment_id = 'PAY1';  
SELECT amount FROM payment_contract WHERE payment_id = 'PAY1';  
ROLLBACK;  
SELECT amount FROM payment_contract WHERE payment_id = 'PAY1';
```

The first inspected amount is 60; after rollback it is 50. The initial creation and insert were explicitly committed in the first transaction and remain. The later update belonged to the cancelled transaction. The check allows zero deliberately; a strict > 0 check would encode a different rule.

Changed retry: explained review

```
CREATE TEMP TABLE stock_retry (  
  product_id text PRIMARY KEY,  
  units integer NOT NULL CHECK (units >= 0)  
);  
INSERT INTO stock_retry VALUES ('ONE', 5);  
BEGIN;  
UPDATE stock_retry SET units = 4 WHERE product_id = 'ONE';  
COMMIT;  
SELECT * FROM stock_retry;
```

The result is ONE/4. A second INSERT with product_id ONE fails because the primary key already exists. That failure does not remove or overwrite the original row. The committed update is outside a later transaction's rollback scope; reversing it would require another deliberate change, backed by the intended business rule.

DE03-13 | Indexes and EXPLAIN basics

First practice: explained review

```
SELECT order_no, amount
FROM plan_orders
WHERE order_no = 4321;
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_no, amount
FROM plan_orders
WHERE order_no = 4321;
```

The data result is one row, 4321/4321. The plan commonly uses the order_no index, but your evidence should describe the actual plan you observed. Compare estimated rows to actual rows and name the scan. No fixed millisecond threshold is required. A correct answer admits configuration-dependent plan details.

Changed retry: explained review

```
SELECT COUNT(*) AS matches
FROM plan_orders
WHERE customer_no = 21;
```

```
EXPLAIN
SELECT order_no
FROM plan_orders
WHERE customer_no = 21;
```

There are 100 matches because remainders repeat once per hundred generated integers. The existing index organises order_no, not the customer_no expression. The planner commonly uses a sequential scan for this query in the fixture. The next engineering step would be to assess a real workload before proposing another index, not to index every column automatically.

DE03-14 | Views and reusable database logic

First practice: explained review

```
SELECT order_id, order_gross
FROM order_totals_view
WHERE is_paid = FALSE
ORDER BY order_id;
```

```
SELECT COUNT(*) AS unpaid_orders, SUM(order_gross) AS unpaid_gross
FROM order_totals_view
WHERE is_paid = FALSE;
```

O102/35 and O106/0 remain. There are two unpaid orders and gross 35. Losing empty O106 would leave the same amount but a wrong count and incomplete record coverage. The view must preserve its one-row-per-order contract for this query to be dependable.

Changed retry: explained review

```
BEGIN;
UPDATE view_source SET units = 4 WHERE code = 'A';
SELECT * FROM view_demo;
ROLLBACK;
SELECT * FROM view_demo;
```

Inside the transaction the result is A/4/40. After rollback it returns to A/3/30. The view reads the source state visible to that session. It did not become a snapshot of 40 merely because you inspected it once.

DE03-15 | Integrated unseen SQL challenge

First practice: explained review

A correct order-grain preparation is:

```
WITH items AS
  (SELECT order_id, SUM(quantity * unit_price) AS item_gross
   FROM course.challenge_items
   GROUP BY order_id)
SELECT o.order_id, o.customer_id, o.is_paid,
       COALESCE(i.item_gross, 0) AS item_gross, o.shipping_fee,
       COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross
FROM course.challenge_orders AS o
LEFT JOIN items AS i ON o.order_id = i.order_id
ORDER BY o.order_id;
```

Q301 is 70 items + 10 fee = 80. Q302 is 30 + 10 = 40. Q303 is 80 + 0 = 80. Q304 is empty but has a fee 5, so it is 0 + 5 = 5. Four unique orders total 205: items 180 plus fees 25. Treating every empty order as total zero would lose Q304's real fee.

For the region and rank questions, name the prepared result and add the customer match at order grain:

```
WITH items AS
  (SELECT order_id, SUM(quantity * unit_price) AS amount
   FROM course.challenge_items
   GROUP BY order_id),
  prepared AS
  (SELECT o.order_id, c.region,
         COALESCE(i.amount, 0) + o.shipping_fee AS order_gross
   FROM course.challenge_orders AS o
   LEFT JOIN items AS i ON o.order_id = i.order_id
   LEFT JOIN course.challenge_customers AS c ON o.customer_id = c.customer_id),
  ranked AS
  (SELECT *,
         RANK() OVER (PARTITION BY region
                     ORDER BY order_gross DESC) AS value_rank
   FROM prepared)
SELECT order_id, COALESCE(region,
                          'Unknown') AS region_group,
       order_gross
FROM ranked
WHERE value_rank = 1
ORDER BY region_group, order_id;
```

East's Q301 and Q303 both remain at 80. West's Q302 is 40, and Unknown's Q304 is 5. Region summaries are East two orders/160, West one/40 and Unknown one/5. C12 is the customer with no orders. SUM(DISTINCT shipping_fee) over order headers gives only 15 rather than 25 because the genuine fee of 10 occurs on two different orders. Removing equal amount values is not a valid order-level deduplication rule.

If the SQL is correct only after consulting this answer, retain that first attempt as assisted. The changed delivery below tests whether you can now reproduce the reasoning independently.

Changed retry: explained review

```
WITH items AS
  (SELECT order_id, SUM(quantity * unit_price) AS amount
   FROM course.challenge_retry_items
   GROUP BY order_id),
  prepared AS
  (SELECT o.order_id, c.region,
         COALESCE(i.amount, 0) + o.shipping_fee AS order_gross
   FROM course.challenge_retry_orders AS o
   LEFT JOIN items AS i ON o.order_id = i.order_id
   LEFT JOIN course.challenge_retry_customers AS c ON o.customer_id = c.customer_id)
SELECT COALESCE(region, 'Unknown') AS region_group, COUNT(*) AS orders, SUM(order_gross) AS gross
FROM prepared
GROUP BY region
ORDER BY region_group;
```

R401 is $40 + 6 = 46$; R402 is $50 + 6 = 56$; R403 is empty with fee zero, total zero. East has two orders/102; Unknown has one/0. Three orders total 102, with items 90 and fees 12. The all-order mean is 34. Averaging East's 51 and Unknown's 0 equally would give 25.5 because it ignores their unequal counts. D11 and D12 have no orders.

DE03-16 | Timed SQL interview set and explanation

First practice: explained review

For latest known-customer sales:

```
WITH ranked AS
  (SELECT customer_id, order_id, ordered_on,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                             ORDER BY ordered_on DESC,
                             order_id DESC) AS rn
   FROM course.sales
   WHERE customer_id IS NOT NULL)
SELECT customer_id, order_id, ordered_on
FROM ranked
WHERE rn = 1
ORDER BY customer_id;
```

C01 gets 0009, C02 gets 0011, C03 gets 0013 and C04 gets 0012. Excluding null identity before partitioning avoids pretending all unknown people are one customer. A deterministic ID tie-breaker is part of the rule, although this fixture's latest dates do not require it for those customers.

For the grouped question:

```
SELECT category, COUNT(*) AS sale_rows,
       SUM(quantity * unit_price) AS gross_total
FROM course.sales
GROUP BY category
HAVING COUNT(*) >= 2
AND SUM(quantity * unit_price) > 150
ORDER BY category;
```

Only Paper, three sales/184, qualifies. Writing has three sales but only 140. Bags has 250 but only one sale. The rule needs both conditions true. The all-order query is the CTE pattern from DE03-07: six unique orders with gross values 149, 35, 100, 265, 115 and 0, total 664; independent inputs total 614 + 50.

COUNT(is_paid) counts six known statuses in course.sales, including two FALSE values; paid rows number four. DISTINCT compares selected whole rows, and SUM(DISTINCT shipping_fee) confuses repeated amounts with record identity. A good explanation points to the table's key and measure grain, rather than saying “duplicates are bad” without defining a duplicate.

Score each answer for correct result, preserved edge cases, an independent check and a clear explanation. If any financial total is multiplied or an unknown status is asserted as unpaid, repair that critical meaning error before calling the set ready, regardless of speed.

Changed retry: explained review

```
WITH ranked AS
  (SELECT order_id, category, quantity * unit_price AS gross,
         ROW_NUMBER() OVER (PARTITION BY category
                           ORDER BY quantity * unit_price DESC,
                           order_id) AS rn
   FROM course.practice_sales)
SELECT order_id, category, gross
FROM ranked
WHERE rn = 1
ORDER BY category;

SELECT category, SUM(quantity * unit_price) AS paid_gross
FROM course.practice_sales
WHERE is_paid = TRUE
GROUP BY category
HAVING SUM(quantity * unit_price) > 50
ORDER BY category;
```

The highest sales are Bags P104/120, Paper P102/80 and Writing P101/18. The paid-only threshold result is Paper/80; P104 is unknown-paid, so its larger amount is excluded by the stated rule. WHERE chooses input rows before grouping and window evaluation; filtering rn belongs in the outer query. The challenge_retry total is 102 across three orders, giving mean 34, with items 90 plus fees 12 as an independent check.